

Translated

From: e-maxx.ru

Binary Exponentiation

Binary exponentiation (also known as exponentiation by squaring) is a trick which allows to calculate a^n using only $O(\log n)$ multiplications (instead of $O(n)$ multiplications required by the naive approach).

It also has important applications in many tasks unrelated to arithmetic, since it can be used with any operations that have the property of **associativity**:

$$(X \cdot Y) \cdot Z = X \cdot (Y \cdot Z)$$

Most obviously this applies to modular multiplication, to multiplication of matrices and to other problems which we will discuss below.

Algorithm

Raising a to the power of n is expressed naively as multiplication by a done $n - 1$ times: $a^n = a \cdot a \cdot \dots \cdot a$. However, this approach is not practical for large a or n .

$$a^{b+c} = a^b \cdot a^c \text{ and } a^{2b} = a^b \cdot a^b = (a^b)^2.$$

The idea of binary exponentiation is, that we split the work using the binary representation of the exponent.

Let's write n in base 2, for example:

$$3^{13} = 3^{1101_2} = 3^8 \cdot 3^4 \cdot 3^1$$

Since the number n has exactly $\lfloor \log_2 n \rfloor + 1$ digits in base 2, we only need to perform $O(\log n)$ multiplications, if we know the powers $a^1, a^2, a^4, a^8, \dots, a^{2^{\lfloor \log n \rfloor}}$.

So we only need to know a fast way to compute those. Luckily this is very easy, since an element in the sequence is just the square of the previous element.

$$3^1 = 3$$

$$3^2 = (3^1)^2 = 3^2 = 9$$

$$3^4 = (3^2)^2 = 9^2 = 81$$

$$3^8 = (3^4)^2 = 81^2 = 6561$$

So to get the final answer for 3^{13} , we only need to multiply three of them (skipping 3^2 because the corresponding bit in n is not set): $3^{13} = 6561 \cdot 81 \cdot 3 = 1594323$

The final complexity of this algorithm is $O(\log n)$: we have to compute $\log n$ powers of a , and then have to do at most $\log n$ multiplications to get the final answer from them.

The following recursive approach expresses the same idea:

$$a^n = \begin{cases} 1 & \text{if } n == 0 \\ (a^{\frac{n}{2}})^2 & \text{if } n > 0 \text{ and } n \text{ even} \\ (a^{\frac{n-1}{2}})^2 \cdot a & \text{if } n > 0 \text{ and } n \text{ odd} \end{cases}$$

Implementation

First the recursive approach, which is a direct translation of the recursive formula:

```
long long binpow(long long a, long long b) {
    if (b == 0)
        return 1;
    long long res = binpow(a, b / 2);
    if (b % 2)
        return res * res * a;
    else
        return res * res;
}
```

The second approach accomplishes the same task without recursion. It computes all the powers in a loop, and multiplies the ones with the corresponding set bit in n . Although the complexity of both approaches is identical, this approach will be faster in practice since we don't have the overhead of the recursive calls.

```
long long binpow(long long a, long long b) {
    long long res = 1;
    while (b > 0) {
        if (b & 1)
            res = res * a;
        a = a * a;
        b >>= 1;
    }
    return res;
}
```

Applications

Effective computation of large exponents modulo a number

Problem: Compute $x^n \bmod m$. This is a very common operation. For instance it is used in computing the [modular multiplicative inverse](#).

Solution: Since we know that the modulo operator doesn't interfere with multiplications ($a \cdot b \equiv (a \bmod m) \cdot (b \bmod m) \pmod m$), we can directly use the same code, and just replace every multiplication with a modular multiplication:

```
long long binpow(long long a, long long b, long long m) {
    a %= m;
    long long res = 1;
    while (b > 0) {
        if (b & 1)
            res = res * a % m;
        a = a * a % m;
        b >>= 1;
    }
    return res;
}
```

Note: It's possible to speed this algorithm for large $b \gg m$. If m is a prime number $x^n \equiv x^{n \bmod (m-1)} \pmod m$ for prime m , and $x^n \equiv x^{n \bmod \phi(m)} \pmod m$ for

composite m . This follows directly from Fermat's little theorem and Euler's theorem, see the article about [Modular Inverses](#) for more details.

Effective computation of Fibonacci numbers

Problem: Compute n -th Fibonacci number F_n .

Solution: For more details, see the [Fibonacci Number article](#). We will only go through an overview of the algorithm. To compute the next Fibonacci number, only the two previous ones are needed, as $F_n = F_{n-1} + F_{n-2}$. We can build a 2×2 matrix that describes this transformation: the transition from F_i and F_{i+1} to F_{i+1} and F_{i+2} . For example, applying this transformation to the pair F_0 and F_1 would change it into F_1 and F_2 . Therefore, we can raise this transformation matrix to the n -th power to find F_n in time complexity $O(\log n)$.

Applying a permutation k times

Problem: You are given a sequence of length n . Apply to it a given permutation k times.

Solution: Simply raise the permutation to k -th power using binary exponentiation, and then apply it to the sequence. This will give you a time complexity of $O(n \log k)$.

```
vector<int> applyPermutation(vector<int> sequence, vector<int>
permutation) {
    vector<int> newSequence(sequence.size());
    for(int i = 0; i < sequence.size(); i++) {
        newSequence[i] = sequence[permutation[i]];
    }
    return newSequence;
}

vector<int> permute(vector<int> sequence, vector<int> permutation, long
long k) {
    while (k > 0) {
        if (k & 1) {
            sequence = applyPermutation(sequence, permutation);
        }
        permutation = applyPermutation(permutation, permutation);
        k >>= 1;
    }
}
```

```
    return sequence;
}
```

Note: This task can be solved more efficiently in linear time by building the permutation graph and considering each cycle independently. You could then compute k modulo the size of the cycle and find the final position for each number which is part of this cycle.

Fast application of a set of geometric operations to a set of points

Problem: Given n points p_i , apply m transformations to each of these points. Each transformation can be a shift, a scaling or a rotation around a given axis by a given angle. There is also a "loop" operation which applies a given list of transformations k times ("loop" operations can be nested). You should apply all transformations faster than $O(n \cdot \text{length})$, where length is the total number of transformations to be applied (after unrolling "loop" operations).

Solution: Let's look at how the different types of transformations change the coordinates:

- Shift operation: adds a different constant to each of the coordinates.
- Scaling operation: multiplies each of the coordinates by a different constant.
- Rotation operation: the transformation is more complicated (we won't go in details here), but each of the new coordinates still can be represented as a linear combination of the old ones.

As you can see, each of the transformations can be represented as a linear operation on the coordinates. Thus, a transformation can be written as a 4×4 matrix of the form:

$$\begin{pmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{pmatrix}$$

that, when multiplied by a vector with the old coordinates and a unit gives a new vector with the new coordinates and a unit:

$$(x \quad y \quad z \quad 1) \cdot \begin{pmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{pmatrix} = (x' \quad y' \quad z' \quad 1)$$

(Why introduce a fictitious fourth coordinate, you ask? That is the beauty of [homogeneous coordinates](#), which find great application in computer graphics. Without this, it would not be possible to implement affine operations like the shift operation as a single matrix multiplication, as it requires us to *add* a constant to the coordinates. The affine transformation becomes a linear transformation in the higher dimension!)

Here are some examples of how transformations are represented in matrix form:

- Shift operation: shift x coordinate by 5, y coordinate by 7 and z coordinate by 9.

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 5 & 7 & 9 & 1 \end{pmatrix}$$

- Scaling operation: scale the x coordinate by 10 and the other two by 5.

$$\begin{pmatrix} 10 & 0 & 0 & 0 \\ 0 & 5 & 0 & 0 \\ 0 & 0 & 5 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

- Rotation operation: rotate θ degrees around the x axis following the right-hand rule (counter-clockwise direction).

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Now, once every transformation is described as a matrix, the sequence of transformations can be described as a product of these matrices, and a "loop" of k repetitions can be described as the matrix raised to the power of k (which can be calculated using binary exponentiation in $O(\log k)$). This way, the matrix which represents all transformations can be calculated first in $O(m \log k)$, and then it can be applied to each of the n points in $O(n)$ for a total complexity of $O(n + m \log k)$.

Number of paths of length k in a graph

Problem: Given a directed unweighted graph of n vertices, find the number of paths of length k from any vertex u to any other vertex v .

Solution: This problem is considered in more detail in [a separate article](#). The algorithm consists of raising the adjacency matrix M of the graph (a matrix where $m_{ij} = 1$ if there is an edge from i to j , or 0 otherwise) to the k -th power. Now m_{ij} will be the number of paths of length k from i to j . The time complexity of this solution is $O(n^3 \log k)$.

Note: In that same article, another variation of this problem is considered: when the edges are weighted and it is required to find the minimum weight path containing exactly k edges. As shown in that article, this problem is also solved by exponentiation of the adjacency matrix. The matrix would have the weight of the edge from i to j , or ∞ if there is no such edge. Instead of the usual operation of multiplying two matrices, a modified one should be used: instead of multiplication, both values are added, and instead of a summation, a minimum is taken. That is: $\text{result}_{ij} = \min_{1 \leq k \leq n} (a_{ik} + b_{kj})$.

Variation of binary exponentiation: multiplying two numbers modulo m

Problem: Multiply two numbers a and b modulo m . a and b fit in the built-in data types, but their product is too big to fit in a 64-bit integer. The idea is to compute $a \cdot b \pmod{m}$ without using bignum arithmetics.

Solution: We simply apply the binary construction algorithm described above, only performing additions instead of multiplications. In other words, we have "expanded" the multiplication of two numbers to $O(\log m)$ operations of addition and multiplication by two (which, in essence, is an addition).

$$a \cdot b = \begin{cases} 0 & \text{if } a = 0 \\ 2 \cdot \frac{a}{2} \cdot b & \text{if } a > 0 \text{ and } a \text{ even} \\ 2 \cdot \frac{a-1}{2} \cdot b + b & \text{if } a > 0 \text{ and } a \text{ odd} \end{cases}$$

Note: You can solve this task in a different way by using floating-point operations. First compute the expression $\frac{a \cdot b}{m}$ using floating-point numbers and cast it to an unsigned integer q . Subtract $q \cdot m$ from $a \cdot b$ using unsigned integer arithmetics and take it modulo m to find the answer. This solution looks rather unreliable, but it is very fast, and very easy to implement. See [here](#) for more information.

Translated

From: e-maxx.ru

Euclidean algorithm for computing the greatest common divisor

Given two non-negative integers a and b , we have to find their **GCD** (greatest common divisor), i.e. the largest number which is a divisor of both a and b . It's commonly denoted by $\gcd(a, b)$. Mathematically it is defined as:

$$\gcd(a, b) = \max \{k > 0 : (k \mid a) \text{ and } (k \mid b)\}$$

(here the symbol " $\mid$ " denotes divisibility, i.e. " $k \mid a$ " means " k divides a ")

When one of the numbers is zero, while the other is non-zero, their greatest common divisor, by definition, is the second number. When both numbers are zero, their greatest common divisor is undefined (it can be any arbitrarily large number), but it is convenient to define it as zero as well to preserve the associativity of $\gcd$. Which gives us a simple rule: if one of the numbers is zero, the greatest common divisor is the other number.

The Euclidean algorithm, discussed below, allows to find the greatest common divisor of two numbers a and b in $O(\log \min(a, b))$. Since the function is **associative**, to find the GCD of **more than two numbers**, we can do $\gcd(a, b, c) = \gcd(a, \gcd(b, c))$ and so forth.

The algorithm was first described in Euclid's "Elements" (circa 300 BC), but it is possible that the algorithm has even earlier origins.

Algorithm

Originally, the Euclidean algorithm was formulated as follows: subtract the smaller number from the larger one until one of the numbers is zero. Indeed, if g divides a and b , it also divides $a - b$. On the other hand, if g divides $a - b$ and b , then it also divides

$a = b + (a - b)$, which means that the sets of the common divisors of $\{a, b\}$ and $\{b, a - b\}$ coincide.

Note that a remains the larger number until b is subtracted from it at least $\lfloor \frac{a}{b} \rfloor$ times.

Therefore, to speed things up, $a - b$ is substituted with $a - \lfloor \frac{a}{b} \rfloor b = a \bmod b$. Then the algorithm is formulated in an extremely simple way:

$$\text{gcd}(a, b) = \begin{cases} a, & \text{if } b = 0 \\ \text{gcd}(b, a \bmod b), & \text{otherwise.} \end{cases}$$

Implementation

```
int gcd (int a, int b) {
    if (b == 0)
        return a;
    else
        return gcd (b, a % b);
}
```

Using the ternary operator in C++, we can write it as a one-liner.

```
int gcd (int a, int b) {
    return b ? gcd (b, a % b) : a;
}
```

And finally, here is a non-recursive implementation:

```
int gcd (int a, int b) {
    while (b) {
        a %= b;
        swap(a, b);
    }
    return a;
}
```

Note that since C++17, `gcd` is implemented as a [standard function](#) in C++.

Time Complexity

The running time of the algorithm is estimated by Lamé's theorem, which establishes a surprising connection between the Euclidean algorithm and the Fibonacci sequence:

If $a > b \geq 1$ and $b < F_n$ for some n , the Euclidean algorithm performs at most $n - 2$ recursive calls.

Moreover, it is possible to show that the upper bound of this theorem is optimal. When $a = F_n$ and $b = F_{n-1}$, $\gcd(a, b)$ will perform exactly $n - 2$ recursive calls. In other words, consecutive Fibonacci numbers are the worst case input for Euclid's algorithm.

Given that Fibonacci numbers grow exponentially, we get that the Euclidean algorithm works in $O(\log \min(a, b))$.

Another way to estimate the complexity is to notice that $a \bmod b$ for the case $a \geq b$ is at least 2 times smaller than a , so the larger number is reduced at least in half on each iteration of the algorithm. Applying this reasoning to the case when we compute the GCD of the set of numbers $a_1, \dots, a_n \leq C$, this also allows us to estimate the total runtime as $O(n + \log C)$, rather than $O(n \log C)$, since every non-trivial iteration of the algorithm reduces the current GCD candidate by at least a factor of 2.

Least common multiple

Calculating the least common multiple (commonly denoted **LCM**) can be reduced to calculating the GCD with the following simple formula:

$$\text{lcm}(a, b) = \frac{a \cdot b}{\gcd(a, b)}$$

Thus, LCM can be calculated using the Euclidean algorithm with the same time complexity:

A possible implementation, that cleverly avoids integer overflows by first dividing a with the GCD, is given here:

```
int lcm (int a, int b) {
    return a / gcd(a, b) * b;
}
```

Binary GCD

The Binary GCD algorithm is an optimization to the normal Euclidean algorithm.

The slow part of the normal algorithm are the modulo operations. Modulo operations, although we see them as $O(1)$, are a lot slower than simpler operations like addition, subtraction or bitwise operations. So it would be better to avoid those.

It turns out, that you can design a fast GCD algorithm that avoids modulo operations. It's based on a few properties:

- If both numbers are even, then we can factor out a two of both and compute the GCD of the remaining numbers: $\text{gcd}(2a, 2b) = 2 \text{gcd}(a, b)$.
- If one of the numbers is even and the other one is odd, then we can remove the factor 2 from the even one: $\text{gcd}(2a, b) = \text{gcd}(a, b)$ if b is odd.
- If both numbers are odd, then subtracting one number of the other one will not change the GCD: $\text{gcd}(a, b) = \text{gcd}(b, a - b)$

Using only these properties, and some fast bitwise functions from GCC, we can implement a fast version:

```
int gcd(int a, int b) {
    if (!a || !b)
        return a | b;
    unsigned shift = __builtin_ctz(a | b);
    a >>= __builtin_ctz(a);
    do {
        b >>= __builtin_ctz(b);
        if (a > b)
            swap(a, b);
        b -= a;
    } while (b);
    return a << shift;
}
```

Notice, that such an optimization is usually not necessary, and most programming languages already have a GCD function in their standard libraries. E.g. C++17 has such a function `std::gcd` in the `numeric` header.

Translated

From: e-maxx.ru

Extended Euclidean Algorithm

While the [Euclidean algorithm](#) calculates only the greatest common divisor (GCD) of two integers a and b , the extended version also finds a way to represent GCD in terms of a and b , i.e. coefficients x and y for which:

$$a \cdot x + b \cdot y = \gcd(a, b)$$

It's important to note that by [Bézout's identity](#) we can always find such a representation. For instance, $\gcd(55, 80) = 5$, therefore we can represent 5 as a linear combination with the terms 55 and 80: $55 \cdot 3 + 80 \cdot (-2) = 5$

A more general form of that problem is discussed in the article about [Linear Diophantine Equations](#). It will build upon this algorithm.

Algorithm

We will denote the GCD of a and b with g in this section.

The changes to the original algorithm are very simple. If we recall the algorithm, we can see that the algorithm ends with $b = 0$ and $a = g$. For these parameters we can easily find coefficients, namely $g \cdot 1 + 0 \cdot 0 = g$.

Starting from these coefficients $(x, y) = (1, 0)$, we can go backwards up the recursive calls. All we need to do is to figure out how the coefficients x and y change during the transition from (a, b) to $(b, a \bmod b)$.

Let us assume we found the coefficients (x_1, y_1) for $(b, a \bmod b)$:

$$b \cdot x_1 + (a \bmod b) \cdot y_1 = g$$

and we want to find the pair (x, y) for (a, b) :

$$a \cdot x + b \cdot y = g$$

We can represent $a \bmod b$ as:

$$a \bmod b = a - \lfloor \frac{a}{b} \rfloor \cdot b$$

Substituting this expression in the coefficient equation of (x_1, y_1) gives:

$$g = b \cdot x_1 + (a \bmod b) \cdot y_1 = b \cdot x_1 + (a - \lfloor \frac{a}{b} \rfloor \cdot b) \cdot y_1$$

and after rearranging the terms:

$$g = a \cdot y_1 + b \cdot (x_1 - y_1 \cdot \lfloor \frac{a}{b} \rfloor)$$

We found the values of x and y :

$$\begin{cases} x = y_1 \\ y = x_1 - y_1 \cdot \lfloor \frac{a}{b} \rfloor \end{cases}$$

Implementation

```
int gcd(int a, int b, int& x, int& y) {
    if (b == 0) {
```

```

        x = 1;
        y = 0;
        return a;
    }
    int x1, y1;
    int d = gcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - y1 * (a / b);
    return d;
}

```

The recursive function above returns the GCD and the values of coefficients to `x` and `y` (which are passed by reference to the function).

This implementation of extended Euclidean algorithm produces correct results for negative integers as well.

Iterative version

It's also possible to write the Extended Euclidean algorithm in an iterative way. Because it avoids recursion, the code will run a little bit faster than the recursive one.

```

int gcd(int a, int b, int& x, int& y) {
    x = 1, y = 0;
    int x1 = 0, y1 = 1, a1 = a, b1 = b;
    while (b1) {
        int q = a1 / b1;
        tie(x, x1) = make_tuple(x1, x - q * x1);
        tie(y, y1) = make_tuple(y1, y - q * y1);
        tie(a1, b1) = make_tuple(b1, a1 - q * b1);
    }
    return a1;
}

```

If you look closely at the variables `a1` and `b1`, you can notice that they take exactly the same values as in the iterative version of the normal [Euclidean algorithm](#). So the algorithm will at least compute the correct GCD.

To see why the algorithm computes the correct coefficients, consider that the following invariants hold at any given time (before the while loop begins and at the end of each iteration):

$$x \cdot a + y \cdot b = a_1$$

$$x_1 \cdot a + y_1 \cdot b = b_1$$

Let the values at the end of an iteration be denoted by a prime ('), and assume $q = \frac{a_1}{b_1}$.
From the [Euclidean algorithm](#), we have:

$$a'_1 = b_1$$

$$b'_1 = a_1 - q \cdot b_1$$

For the first invariant to hold, the following should be true:

$$x' \cdot a + y' \cdot b = a'_1 = b_1$$

$$x' \cdot a + y' \cdot b = x_1 \cdot a + y_1 \cdot b$$

Similarly for the second invariant, the following should hold:

$$x'_1 \cdot a + y'_1 \cdot b = a_1 - q \cdot b_1$$

$$x'_1 \cdot a + y'_1 \cdot b = (x - q \cdot x_1) \cdot a + (y - q \cdot y_1) \cdot b$$

By comparing the coefficients of a and b , the update equations for each variable can be derived, ensuring that the invariants are maintained throughout the algorithm.

At the end we know that a_1 contains the GCD, so $x \cdot a + y \cdot b = g$. Which means that we have found the required coefficients.

You can even optimize the code more, and remove the variable a_1 and b_1 from the code, and just reuse a and b . However if you do so, you lose the ability to argue about the invariants.

Translated

From: e-maxx.ru

Linear Diophantine Equation

A Linear Diophantine Equation (in two variables) is an equation of the general form:

$$ax + by = c$$

where a, b, c are given integers, and x, y are unknown integers.

In this article, we consider several classical problems on these equations:

- finding one solution
- finding all solutions
- finding the number of solutions and the solutions themselves in a given interval
- finding a solution with minimum value of $x + y$

The degenerate case

A degenerate case that need to be taken care of is when $a = b = 0$. It is easy to see that we either have no solutions or infinitely many solutions, depending on whether $c = 0$ or not. In the rest of this article, we will ignore this case.

Analytic solution

When $a \neq 0$ and $b \neq 0$, the equation $ax + by = c$ can be equivalently treated as either of the following:

$$ax \equiv c \pmod{b}$$

$$by \equiv c \pmod{a}$$

Without loss of generality, assume that $b \neq 0$ and consider the first equation. When a and b are co-prime, the solution to it is given as

$$x \equiv ca^{-1} \pmod{b},$$

where a^{-1} is the [modular inverse](#) of a modulo b .

When a and b are not co-prime, values of ax modulo b for all integer x are divisible by $g = \gcd(a, b)$, so the solution only exists when c is divisible by g . In this case, one of solutions can be found by reducing the equation by g :

$$(a/g)x \equiv (c/g) \pmod{b/g}.$$

By the definition of g , the numbers a/g and b/g are co-prime, so the solution is given explicitly as

$$\begin{cases} x \equiv (c/g)(a/g)^{-1} \pmod{b/g}, \\ y = \frac{c-ax}{b}. \end{cases}$$

Algorithmic solution

Bézout's lemma (also called Bézout's identity) is a useful result that can be used to understand the following solution.

Let $g = \gcd(a, b)$. Then there exist integers x, y such that $ax + by = g$.

Moreover, g is the least such positive integer that can be written as $ax + by$; all integers of the form $ax + by$ are multiples of g .

To find one solution of the Diophantine equation with 2 unknowns, you can use the [Extended Euclidean algorithm](#). First, assume that a and b are non-negative. When we apply Extended Euclidean algorithm for a and b , we can find their greatest common divisor g and 2 numbers x_g and y_g such that:

$$ax_g + by_g = g$$

If c is divisible by $g = \gcd(a, b)$, then the given Diophantine equation has a solution, otherwise it does not have any solution. The proof is straight-forward: a linear combination of two numbers is divisible by their common divisor.

Now suppose that c is divisible by g , then we have:

$$a \cdot x_g \cdot \frac{c}{g} + b \cdot y_g \cdot \frac{c}{g} = c$$

Therefore one of the solutions of the Diophantine equation is:

$$x_0 = x_g \cdot \frac{c}{g},$$

$$y_0 = y_g \cdot \frac{c}{g}.$$

The above idea still works when a or b or both of them are negative. We only need to change the sign of x_0 and y_0 when necessary.

Finally, we can implement this idea as follows (note that this code does not consider the case $a = b = 0$):

```
int gcd(int a, int b, int& x, int& y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    int x1, y1;
    int d = gcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - y1 * (a / b);
    return d;
}

bool find_any_solution(int a, int b, int c, int &x0, int &y0, int &g) {
```

```

g = gcd(abs(a), abs(b), x0, y0);
if (c % g) {
    return false;
}

x0 *= c / g;
y0 *= c / g;
if (a < 0) x0 = -x0;
if (b < 0) y0 = -y0;
return true;
}

```

Getting all solutions

From one solution (x_0, y_0) , we can obtain all the solutions of the given equation.

Let $g = \gcd(a, b)$ and let x_0, y_0 be integers which satisfy the following:

$$a \cdot x_0 + b \cdot y_0 = c$$

Now, we should see that adding b/g to x_0 , and, at the same time subtracting a/g from y_0 will not break the equality:

$$a \cdot \left(x_0 + \frac{b}{g}\right) + b \cdot \left(y_0 - \frac{a}{g}\right) = a \cdot x_0 + b \cdot y_0 + a \cdot \frac{b}{g} - b \cdot \frac{a}{g} = c$$

Obviously, this process can be repeated again, so all the numbers of the form:

$$x = x_0 + k \cdot \frac{b}{g}$$

$$y = y_0 - k \cdot \frac{a}{g}$$

are solutions of the given Diophantine equation.

Since the equation is linear, all solutions lie on the same line, and by the definition of g this is the set of all possible solutions of the given Diophantine equation.

Finding the number of solutions and the solutions in a given interval

From previous section, it should be clear that if we don't impose any restrictions on the solutions, there would be infinite number of them. So in this section, we add some restrictions on the interval of x and y , and we will try to count and enumerate all the solutions.

Let there be two intervals: $[\min_x; \max_x]$ and $[\min_y; \max_y]$ and let's say we only want to find the solutions in these two intervals.

Note that if a or b is 0 , then the problem only has one solution. We don't consider this case here.

First, we can find a solution which has minimum value of x , such that $x \geq \min_x$. To do this, we first find any solution of the Diophantine equation. Then, we shift this solution to get $x \geq \min_x$ (using what we know about the set of all solutions in previous section). This can be done in $O(1)$. Denote this minimum value of x by l_{x1} .

Similarly, we can find the maximum value of x which satisfies $x \leq \max_x$. Denote this maximum value of x by r_{x1} .

Similarly, we can find the minimum value of y ($y \geq \min_y$) and maximum value of y ($y \leq \max_y$). Denote the corresponding values of x by l_{x2} and r_{x2} .

The final solution is all solutions with x in intersection of $[l_{x1}, r_{x1}]$ and $[l_{x2}, r_{x2}]$. Let denote this intersection by $[l_x, r_x]$.

Following is the code implementing this idea. Notice that we divide a and b at the beginning by g . Since the equation $ax + by = c$ is equivalent to the equation $\frac{a}{g}x + \frac{b}{g}y = \frac{c}{g}$, we can use this one instead and have $\gcd(\frac{a}{g}, \frac{b}{g}) = 1$, which simplifies the formulas.

```
void shift_solution(int & x, int & y, int a, int b, int cnt) {
    x += cnt * b;
```

```

    y -= cnt * a;
}

int find_all_solutions(int a, int b, int c, int minx, int maxx, int
miny, int maxy) {
    int x, y, g;
    if (!find_any_solution(a, b, c, x, y, g))
        return 0;
    a /= g;
    b /= g;

    int sign_a = a > 0 ? +1 : -1;
    int sign_b = b > 0 ? +1 : -1;

    shift_solution(x, y, a, b, (minx - x) / b);
    if (x < minx)
        shift_solution(x, y, a, b, sign_b);
    if (x > maxx)
        return 0;
    int lx1 = x;

    shift_solution(x, y, a, b, (maxx - x) / b);
    if (x > maxx)
        shift_solution(x, y, a, b, -sign_b);
    int rx1 = x;

    shift_solution(x, y, a, b, -(miny - y) / a);
    if (y < miny)
        shift_solution(x, y, a, b, -sign_a);
    if (y > maxy)
        return 0;
    int lx2 = x;

    shift_solution(x, y, a, b, -(maxy - y) / a);
    if (y > maxy)
        shift_solution(x, y, a, b, sign_a);
    int rx2 = x;

    if (lx2 > rx2)
        swap(lx2, rx2);
    int lx = max(lx1, lx2);
    int rx = min(rx1, rx2);

    if (lx > rx)
        return 0;
    return (rx - lx) / abs(b) + 1;
}

```

Once we have l_x and r_x , it is also simple to enumerate through all the solutions. Just need to iterate through $x = l_x + k \cdot \frac{b}{g}$ for all $k \geq 0$ until $x = r_x$, and find the corresponding y values using the equation $ax + by = c$.

Find the solution with minimum value of $x + y$

Here, x and y also need to be given some restriction, otherwise, the answer may become negative infinity.

The idea is similar to previous section: We find any solution of the Diophantine equation, and then shift the solution to satisfy some conditions.

Finally, use the knowledge of the set of all solutions to find the minimum:

$$x' = x + k \cdot \frac{b}{g},$$

$$y' = y - k \cdot \frac{a}{g}.$$

Note that $x + y$ change as follows:

$$x' + y' = x + y + k \cdot \left(\frac{b}{g} - \frac{a}{g} \right) = x + y + k \cdot \frac{b - a}{g}$$

If $a < b$, we need to select smallest possible value of k . If $a > b$, we need to select the largest possible value of k . If $a = b$, all solution will have the same sum $x + y$.

Translated

From: e-maxx.ru

Fibonacci Numbers

The Fibonacci sequence is defined as follows:

$$F_0 = 0, F_1 = 1, F_n = F_{n-1} + F_{n-2}$$

The first elements of the sequence ([OEIS A000045](#)) are:

$$0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, \dots$$

Properties

Fibonacci numbers possess a lot of interesting properties. Here are a few of them:

- Cassini's identity:

$$F_{n-1}F_{n+1} - F_n^2 = (-1)^n$$

This can be proved by induction. A one-line proof by Knuth comes from taking the determinant of the 2x2 matrix form below.

- The "addition" rule:

$$F_{n+k} = F_k F_{n+1} + F_{k-1} F_n$$

- Applying the previous identity to the case $k = n$, we get:

$$F_{2n} = F_n(F_{n+1} + F_{n-1})$$

- From this we can prove by induction that for any positive integer k , F_{nk} is multiple of F_n .
- The inverse is also true: if F_m is multiple of F_n , then m is multiple of n .
- GCD identity:

$$\text{GCD}(F_m, F_n) = F_{\text{GCD}(m, n)}$$

- Fibonacci numbers are the worst possible inputs for Euclidean algorithm (see [Lame's theorem](#) in [Euclidean algorithm](#))

Fibonacci Coding

We can use the sequence to encode positive integers into binary code words. According to Zeckendorf's theorem, any natural number n can be uniquely represented as a sum of Fibonacci numbers:

$$N = F_{k_1} + F_{k_2} + \dots + F_{k_r}$$

such that $k_1 \geq k_2 + 2$, $k_2 \geq k_3 + 2$, $\dots$, $k_r \geq 2$ (i.e.: the representation cannot use two consecutive Fibonacci numbers).

It follows that any number can be uniquely encoded in the Fibonacci coding. And we can describe this representation with binary codes $d_0 d_1 d_2 \dots d_s 1$, where d_i is 1 if F_{i+2} is used in the representation. The code will be appended by a 1 to indicate the end of the code word. Notice that this is the only occurrence where two consecutive 1-bits appear.

$$\begin{array}{rclcl}
 1 & = & F_2 & = & (11)_F \\
 2 & = & F_3 & = & (011)_F \\
 6 & = & 5 + 1 & = & F_5 + F_2 = (10011)_F \\
 8 & = & 8 & = & F_6 = (000011)_F \\
 9 & = & 8 + 1 & = & F_6 + F_2 = (100011)_F \\
 19 & = & 13 + 5 + 1 & = & F_7 + F_5 + F_2 = (1001011)_F
 \end{array}$$

The encoding of an integer n can be done with a simple greedy algorithm:

1. Iterate through the Fibonacci numbers from the largest to the smallest until you find one less than or equal to n .
2. Suppose this number was F_i . Subtract F_i from n and put a 1 in the $i - 2$ position of the code word (indexing from 0 from the leftmost to the rightmost bit).
3. Repeat until there is no remainder.
4. Add a final 1 to the codeword to indicate its end.

To decode a code word, first remove the final 1. Then, if the i -th bit is set (indexing from 0 from the leftmost to the rightmost bit), sum F_{i+2} to the number.

Formulas for the n^{th} Fibonacci number

Closed-form expression

There is a formula known as "Binet's formula", even though it was already known by Moivre:

$$F_n = \frac{\left(\frac{1+\sqrt{5}}{2}\right)^n - \left(\frac{1-\sqrt{5}}{2}\right)^n}{\sqrt{5}}$$

This formula is easy to prove by induction, but it can be deduced with the help of the concept of generating functions or by solving a functional equation.

You can immediately notice that the second term's absolute value is always less than 1, and it also decreases very rapidly (exponentially). Hence the value of the first term alone is "almost" F_n . This can be written strictly as:

$$F_n = \left\lceil \frac{\left(\frac{1+\sqrt{5}}{2}\right)^n}{\sqrt{5}} \right\rceil$$

where the square brackets denote rounding to the nearest integer.

As these two formulas would require very high accuracy when working with fractional numbers, they are of little use in practical calculations.

Fibonacci in linear time

The n -th Fibonacci number can be easily found in $O(n)$ by computing the numbers one by one up to n . However, there are also faster ways, as we will see.

We can start from an iterative approach, to take advantage of the use of the formula $F_n = F_{n-1} + F_{n-2}$, therefore, we will simply precalculate those values in an array. Taking into account the base cases for F_0 and F_1 .

```
int fib(int n) {
    int a = 0;
    int b = 1;
    for (int i = 0; i < n; i++) {
        int tmp = a + b;
        a = b;
        b = tmp;
    }
    return a;
}
```

In this way, we obtain a linear solution, $O(n)$ time, saving all the values prior to n in the sequence.

Matrix form

To go from (F_n, F_{n-1}) to (F_{n+1}, F_n) , we can express the linear recurrence as a 2x2 matrix multiplication:

$$\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} F_n \\ F_{n-1} \end{pmatrix} = \begin{pmatrix} F_n + F_{n-1} \\ F_n \end{pmatrix} = \begin{pmatrix} F_{n+1} \\ F_n \end{pmatrix}$$

This lets us treat iterating the recurrence as repeated matrix multiplication, which has nice properties. In particular,

$$\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n \begin{pmatrix} F_1 \\ F_0 \end{pmatrix} = \begin{pmatrix} F_{n+1} \\ F_n \end{pmatrix}$$

where $F_1 = 1, F_0 = 0$. In fact, since

$$\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} F_2 & F_1 \\ F_1 & F_0 \end{pmatrix}$$

we can use the matrix directly:

$$\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n = \begin{pmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{pmatrix}$$

Thus, in order to find F_n in $O(\log n)$ time, we must raise the matrix to n . (See [Binary exponentiation](#))

```
struct matrix {
    long long mat[2][2];
    matrix friend operator *(const matrix &a, const matrix &b){
        matrix c;
        for (int i = 0; i < 2; i++) {
            for (int j = 0; j < 2; j++) {
                c.mat[i][j] = 0;
                for (int k = 0; k < 2; k++) {
                    c.mat[i][j] += a.mat[i][k] * b.mat[k][j];
                }
            }
        }
        return c;
    }
};

matrix matpow(matrix base, long long n) {
    matrix ans{ {
        {1, 0},
        {0, 1}
    } };
    while (n) {
        if(n&1)
            ans = ans*base;
    }
}
```

```

        base = base*base;
        n >>= 1;
    }
    return ans;
}

long long fib(int n) {
    matrix base{ {
        {1, 1},
        {1, 0}
    } };
    return matpow(base, n).mat[0][1];
}

```

Fast Doubling Method

By expanding the above matrix expression for $n = 2 \cdot k$

$$\begin{pmatrix} F_{2k+1} & F_{2k} \\ F_{2k} & F_{2k-1} \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^{2k} = \begin{pmatrix} F_{k+1} & F_k \\ F_k & F_{k-1} \end{pmatrix}^2$$

we can find these simpler equations:

$$F_{2k+1} = F_{k+1}^2 + F_k^2$$

$$F_{2k} = F_k(F_{k+1} + F_{k-1}) = F_k(2F_{k+1} - F_k)$$

Thus using above two equations Fibonacci numbers can be calculated easily by the following code:

```

pair<int, int> fib (int n) {
    if (n == 0)
        return {0, 1};

    auto p = fib(n >> 1);
    int c = p.first * (2 * p.second - p.first);
    int d = p.first * p.first + p.second * p.second;
    if (n & 1)
        return {d, c + d};
    else

```

```
    return {c, d};
}
```

The above code returns F_n and F_{n+1} as a pair.

Periodicity modulo p

Consider the Fibonacci sequence modulo p . We will prove the sequence is periodic.

Let us prove this by contradiction. Consider the first $p^2 + 1$ pairs of Fibonacci numbers taken modulo p :

$$(F_0, F_1), (F_1, F_2), \dots, (F_{p^2}, F_{p^2+1})$$

There can only be p different remainders modulo p , and at most p^2 different pairs of remainders, so there are at least two identical pairs among them. This is sufficient to prove the sequence is periodic, as a Fibonacci number is only determined by its two predecessors. Hence if two pairs of consecutive numbers repeat, that would also mean the numbers after the pair will repeat in the same fashion.

We now choose two pairs of identical remainders with the smallest indices in the sequence. Let the pairs be (F_a, F_{a+1}) and (F_b, F_{b+1}) . We will prove that $a = 0$. If this was false, there would be two previous pairs (F_{a-1}, F_a) and (F_{b-1}, F_b) , which, by the property of Fibonacci numbers, would also be equal. However, this contradicts the fact that we had chosen pairs with the smallest indices, completing our proof that there is no pre-period (i.e the numbers are periodic starting from F_0).